Inroduction to Function Plots and Animations Using Matplotlib

Until now, we have mainly used NumPy to perform numerical calculations. But numerical results are often difficult to understand just by looking at numbers. Matplotlib allows us to convert those numerical results into graphs. For example, suppose we calculate y=x^2 for several values of x. We may get

(x)	(y=x^2)
0	0
1	1
2	4
3	9
4	16
5	25

Instead of looking at the table, we can plot these values. Main idea **Mathematical function---->Numerical values---->Plot 1. 2D Function Plot** A 2D function plot represents a relationship between two variables:

y=f(x)

The horizontal axis represents x, and the vertical axis represents y. For example:

y=x^2

means: For every value of x, calculate a corresponding value of y.

Matplotlib then places the points

(x0,y0),(x1,y1),.....,(xn,yn)

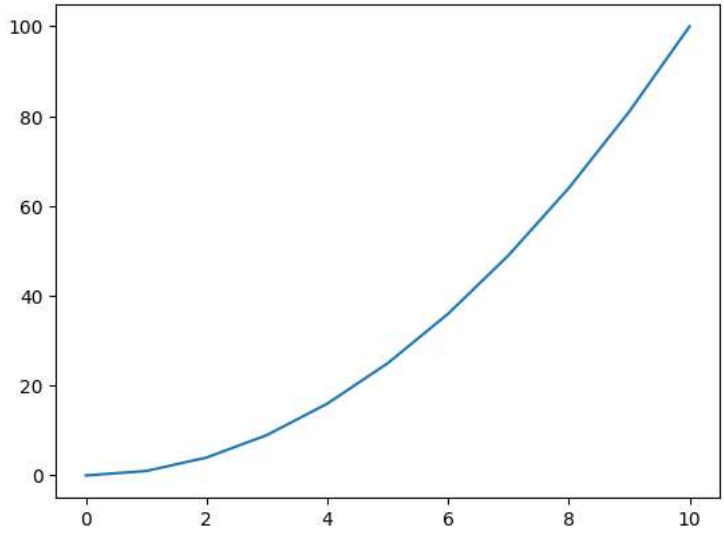
on the coordinate system and connects them to create the curve.

```
import matplotlib.pyplot as plt

lx, ly = [], []

for x in range(11):
    y = x**2
    lx.append(x)
    ly.append(y)

plt.plot(lx, ly)
plt.show()
```



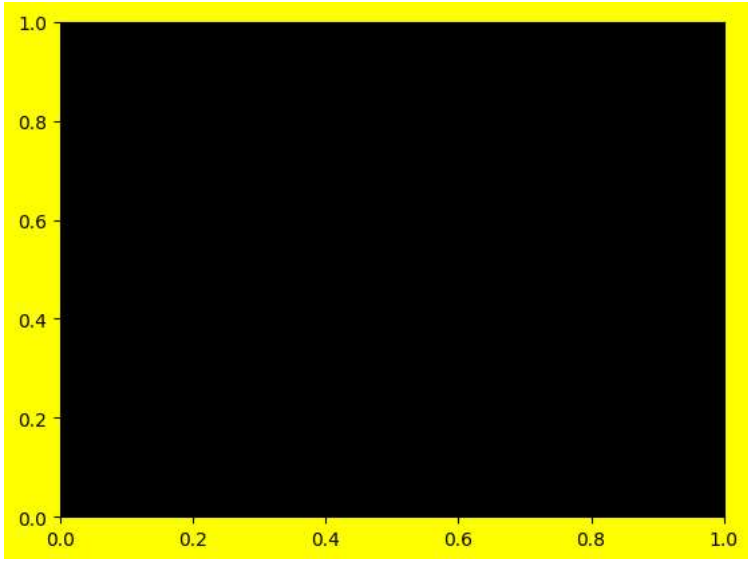
matplotlib → plotting library pyplot → module containing plotting functions plt → short alias(convinient short name for matplotlib.pyplot)

x values-->calculate y = x^2-->store x and y-->plt.plot(x,y)-->graph

2. Objectoriented variant of function plot To understand this first we need to understand two terms

1. Figure (Complete drawing area/Window)
2. Axis (the coordinate system inside the Figure where the actual graph is drawn)

```
fig,ax=plt.subplots()
fig.set_facecolor("yellow")
ax.set_facecolor("black")
```




```
import matplotlib.pyplot as plt
fig=plt.figure() # create a figure object with a fixed coordinate with default values
# gives the default width and height of the drawing area
print(fig.get_figwidth())
print(fig.get_figheight())
# Increasing the width and height of the drawing area
fig.set_figwidth(12)
fig.set_figheight(10)
print(fig.get_figwidth())
print(fig.get_figheight())
```

```
6.4
4.8
12.0
10.0
<Figure size 1200x1000 with 0 Axes>
```

The coordinate data of the figure object cannot be changed. If we multiply the default height and width with the default resolution of 100dpi then we get a drawing area of 640X480 pixels. After setting the height and width to 10 and 12 we get a drawing area of 1200X1000 pixels

Axes A coordinate system can be created using ax=fig.add_subplot() or ax=fig.subplots()

```
import matplotlib.pyplot as plt
ax=fig.subplots()
print(ax.get_position())
```

```
Bbox(x0=0.125, y0=0.10999999999999999, x1=0.9, y1=0.88)
```

Representing a mathematical function Seven steps for representing a mathematical function The book summarizes the representation of a mathematical function in seven steps. chapter 4 Teach them as: **Step 1**

Import NumPy. import numpy as np

Step 2

Import Matplotlib. import matplotlib.pyplot as plt

Step 3

Generate x-values. x = np.linspace(start, stop, number)

Step 4

Define the function. y = f(x)

Step 5

Create a Figure and Axes. fig, ax = plt.subplots()

Step 6

Plot the function. ax.plot(x, y)

Step 7

Display the plot. plt.show()

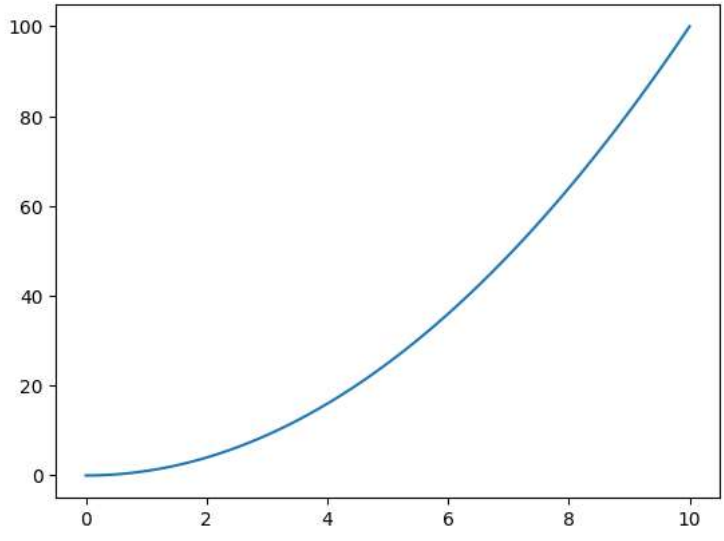
```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 10, 100)

fig, ax = plt.subplots(figsize=(6.4,4.8),dpi=100)

ax.plot(x, x**2)

#fig.show()
plt.show(fig)
```



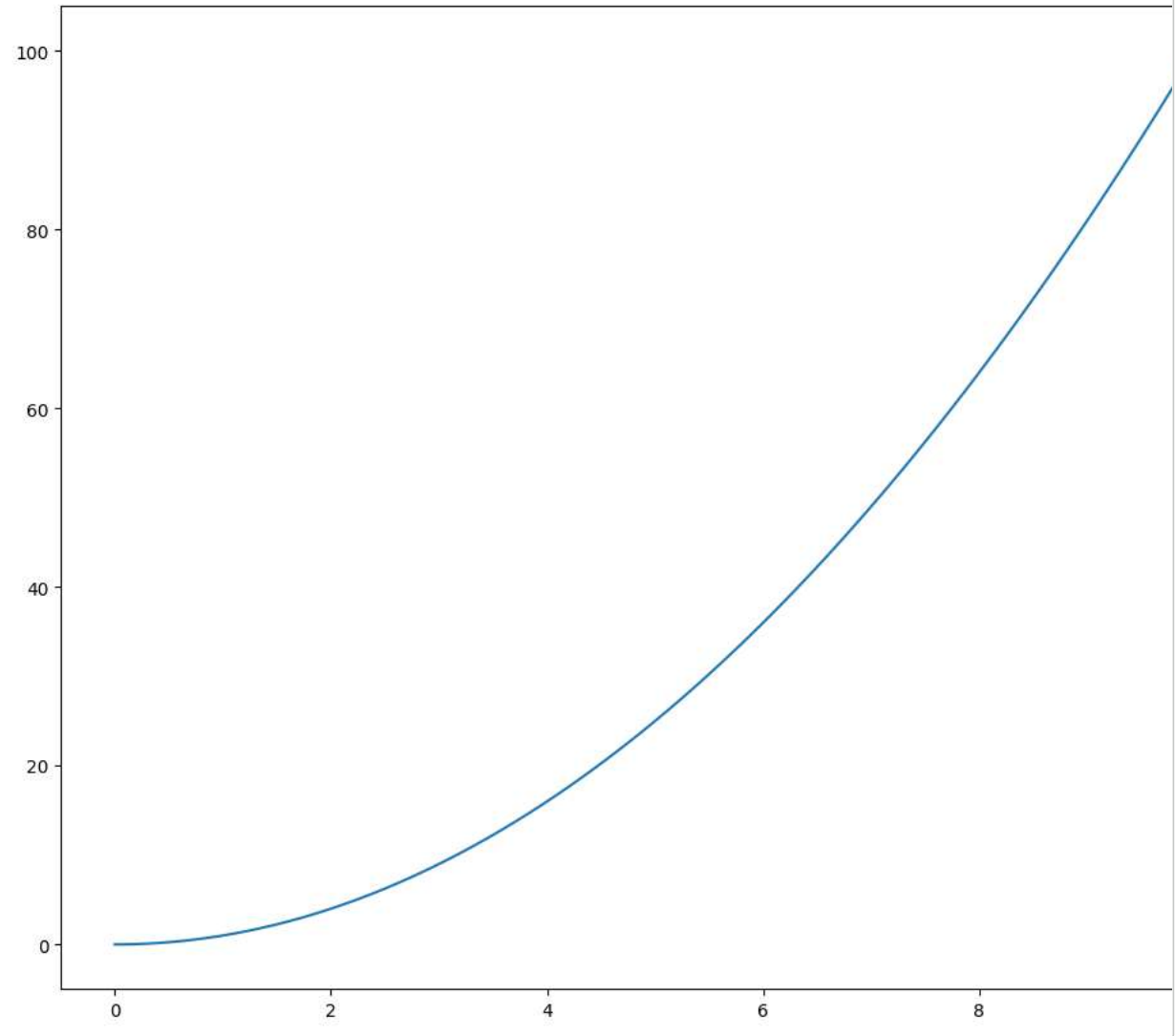

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 10, 100)

fig, ax = plt.subplots(figsize=(12,10),dpi=100)

ax.plot(x, x**2)

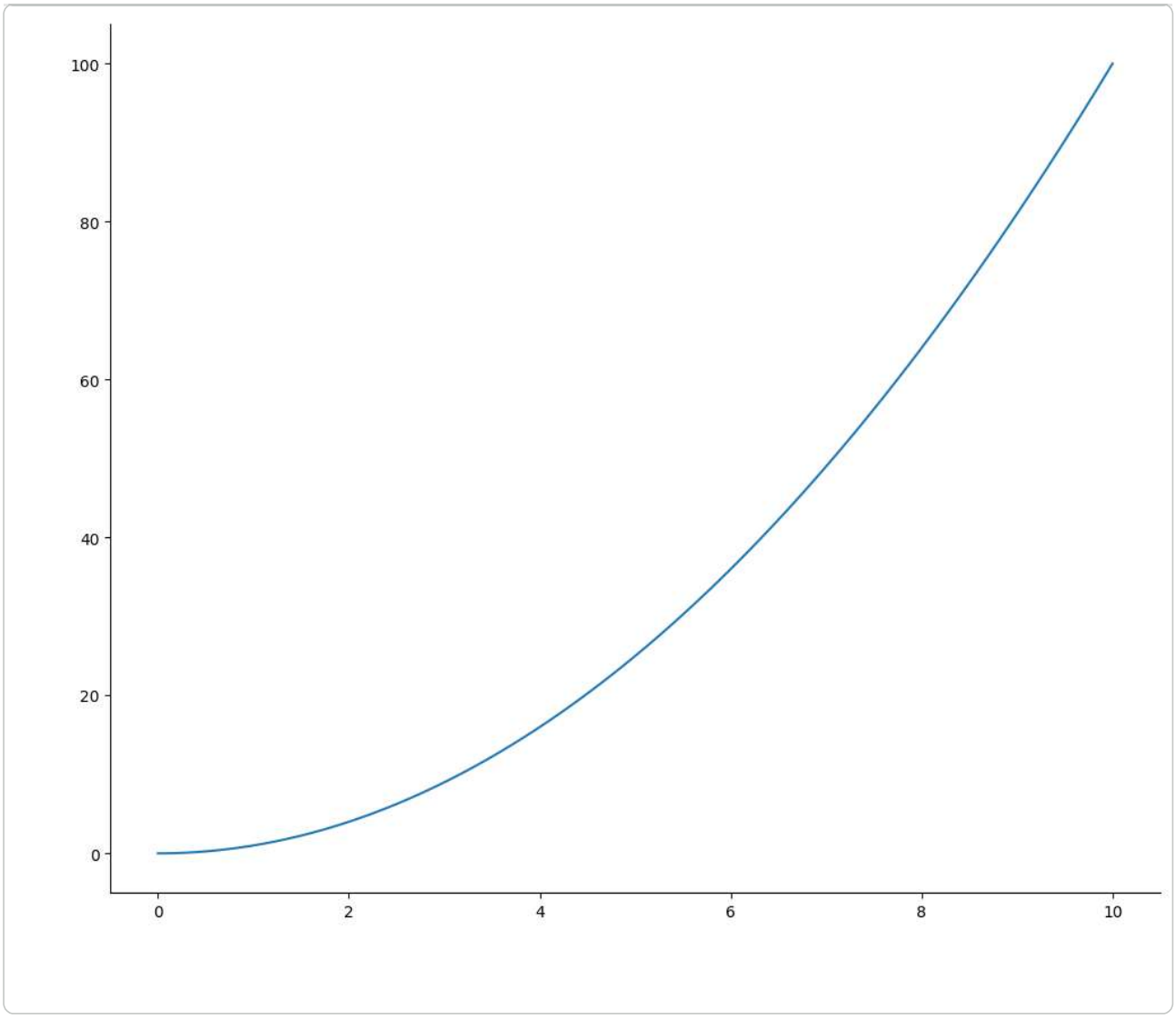
#fig.show()
plt.show(fig)
```




```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 10, 100)

fig, ax = plt.subplots(figsize=(12,10),dpi=100)
ax.spines[['top','right']].set_visible(False) #To hide the display of the top and right spine of the axes object
ax.plot(x, x**2)
fig.savefig("parabel.png")
#fig.show()
plt.show(fig)
```



Note: The fig and ax objects can also be created using figure() and fig.add.subplot() methods, as follows

```
fig=plt.figure()# designing the user interface
ax=fig.add_subplot() # designing the coordinate system
ax=fig.subplots() # for multiple subplots
```

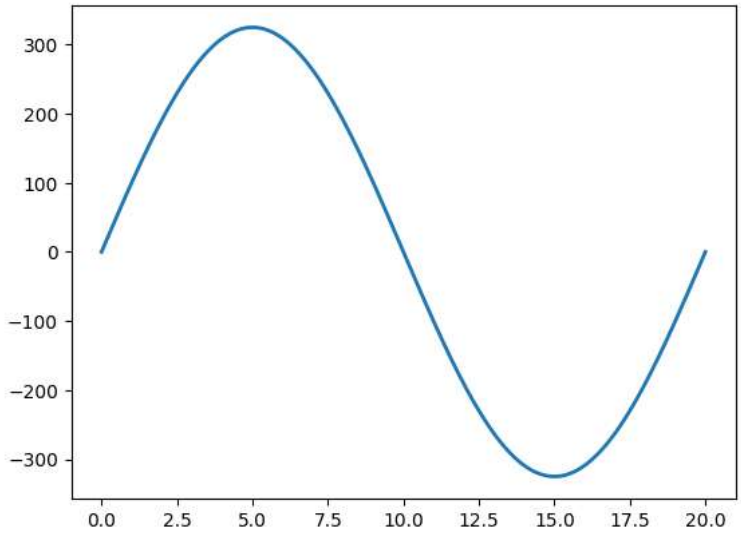
Gridlines To better read function values gridlines are included in function plot The command is **ax.grid(True)**

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 20, 500)
u=325*np.sin(2*np.pi*50*t/1000)

fig, ax = plt.subplots()

ax.plot(t,u,linewidth=2)
#ax.grid(color='black',linestyle='solid',lw=0.5)
#ax.grid(color='black',linestyle='dotted',lw=0.5)
#ax.grid(color='black',linestyle='dashdot',lw=0.5)
#ax.grid(True)
#fig.show()
plt.show(fig)
```

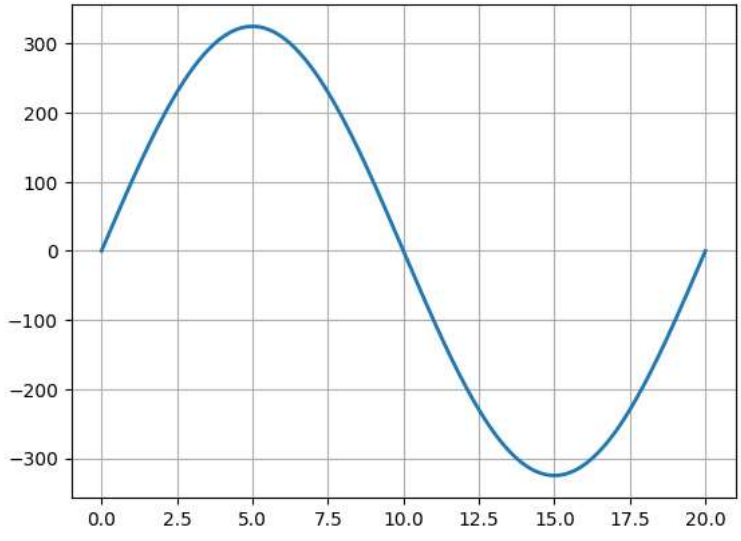



```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 20, 500)
u=325*np.sin(2*np.pi*50*t/1000)

fig, ax = plt.subplots()

ax.plot(t,u,linewidth=2)
#ax.grid(color='black',linestyle='solid',lw=0.5)
#ax.grid(color='black',linestyle='dotted',lw=0.5)
#ax.grid(color='black',linestyle='dashdot',lw=0.5)
ax.grid(True)
#fig.show()
plt.show(fig)
```



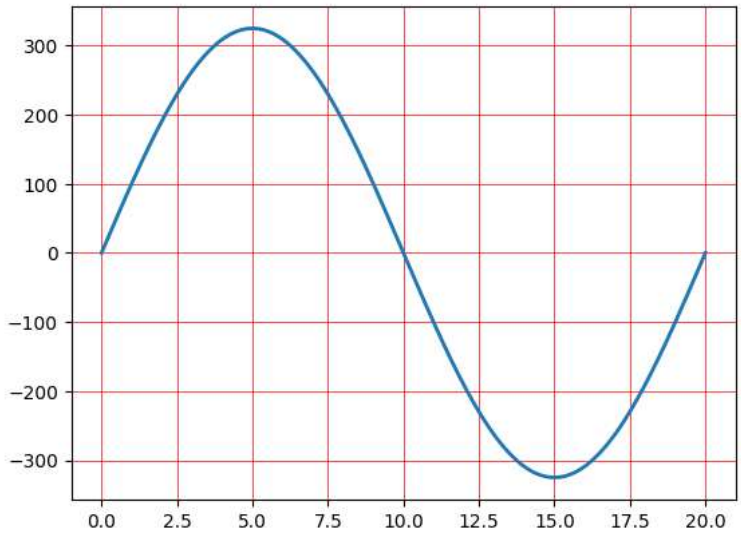

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 20, 500)
u=325*np.sin(2*np.pi*50*t/1000)

fig, ax = plt.subplots()

ax.plot(t,u,linewidth=2)
```

```
ax.grid(color='red',linestyle='solid',lw=0.5)
#ax.grid(color='black',linestyle='dotted',lw=0.5)
#ax.grid(color='black',linestyle='dashdot',lw=0.5)
ax.grid(True)
#fig.show()
plt.show(fig)
```

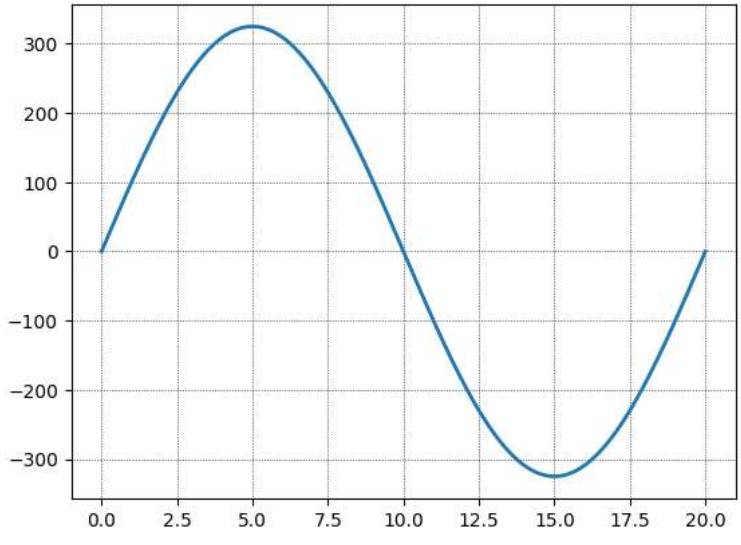



```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 20, 500)
u=325*np.sin(2*np.pi*50*t/1000)

fig, ax = plt.subplots()

ax.plot(t,u,linewidth=2)
#ax.grid(color='red',linestyle='solid',lw=0.5)
ax.grid(color='black',linestyle='dotted',lw=0.5)
#ax.grid(color='black',linestyle='dashdot',lw=0.5)
ax.grid(True)
#fig.show()
plt.show(fig)
```

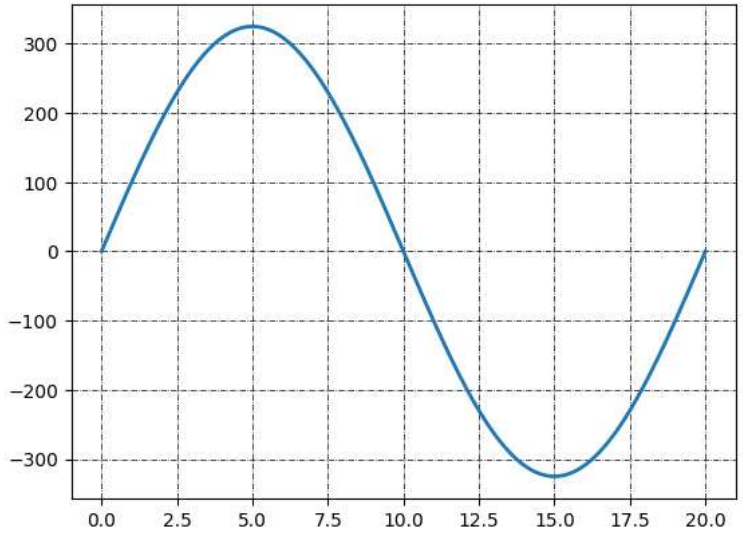



```
import numpy as np
import matplotlib.pyplot as plt
```

```
t = np.linspace(0, 20, 500)
u=325*np.sin(2*np.pi*50*t/1000)

#fig, ax = plt.subplots()
fig=plt.figure() # fig object is created separately
ax=fig.add_subplot()# ax object is created separately

ax.plot(t,u,linewidth=2)
#ax.grid(color='red',linestyle='solid',lw=0.5)
#ax.grid(color='black',linestyle='dotted',lw=0.5)
ax.grid(color='black',ls='dashdot',lw=0.5) # abbreviated notation for linestyle
ax.grid(True)
#fig.show()
plt.show(fig)
```



The fig object is not needed in this program. if we remove it an error occur

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 20, 500)
u=325*np.sin(2*np.pi*50*t/1000)

#fig, ax = plt.subplots()
#fig=plt.figure() # fig object is created separately
ax=fig.add_subplot()# ax object is created separately

ax.plot(t,u,linewidth=2)
#ax.grid(color='red',linestyle='solid',lw=0.5)
#ax.grid(color='black',linestyle='dotted',lw=0.5)
ax.grid(color='black',ls='dashdot',lw=0.5) # abbreviated notation for linestyle
ax.grid(True)
#ax.grid(False) # turn the grid off
#fig.show()
plt.show(fig)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_1513/477691848.py in <cell line: 0>()
      7 #fig, ax = plt.subplots()
      8 #fig=plt.figure() # fig object is created separately
----> 9 ax=fig.add_subplot()# ax object is created separately
     10
     11 ax.plot(t,u,linewidth=2)

NameError: name 'fig' is not defined
```

Next steps: Explain error

Use of gridlines

Read coordinates Compare points **Labels** A graph without labels is scientifically incomplete. looking at the graph several questions may arise:

What does x represent? What does y represent? What is the unit? What does the curve represent? Therefore, we need labels.

For the labels of function plot Matplotlib provides the following methods

ax.set_xlabel()

ax.set_ylabel()

ax.set_title()

ax.legend()

ax.annotate()